

**LAPORAN TAKE HOME TASK
PRAKTIKUM PEMROGRAMAN**

BERORIENTASI OBJEK

MODUL 12



Disusun Oleh:

Ahmad Zaidan 105224044

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA**

2026

I. Pendahuluan

Sistem Informasi Akademik (SIKAD) versi lama memiliki masalah besar pada desain kodenya yang terlalu kaku dan rapuh. Masalah utama berpusat pada kelas SistemKRSManager yang bertindak sebagai "kelas raksasa" karena memborong terlalu banyak tugas sendirian, mulai dari mengelola bisnis KRS, menghitung biaya UKT, menyambung ke database, sampai mencetak dokumen PDF. Selain itu, antarmuka mata kuliah yang lama juga tidak efisien karena memaksa kelas non-praktikum seperti kelas Teori untuk menuliskan fungsi laboratorium yang tidak mereka butuhkan, sehingga program sering mengalami *error*. Keterikatan kode yang terlalu kaku ini membuat sistem kewalahan saat menerima instruksi baru dari Rektorat untuk menambahkan skema biaya program MBKM dan memigrasikan database dari MySQL ke Cloud NoSQL. Jika kode lama dipaksakan berubah, risiko rusaknya fungsi lain di dalam sistem sangatlah besar.

Melalui praktikum rancang ulang kode (*refactoring*) ini, seluruh masalah tersebut diselesaikan dengan menerapkan prinsip SOLID. Tujuan utama dari praktikum ini adalah memecah kelas raksasa dan antarmuka yang cacat menjadi komponen-komponen kecil yang lebih mandiri dan fleksibel. Dengan menerapkan prinsip-prinsip tersebut, praktikum ini ingin membuktikan bahwa sistem SIKAD yang baru bisa menjadi jauh lebih adaptif dalam menghadapi perubahan kebutuhan kampus. Keberhasilan transformasi arsitektur ini diuji secara langsung di dalam kelas Main melalui simulasi pergantian jenis database dan jalur UKT secara dinamis, tanpa perlu mengubah atau merusak logika inti bisnis sistem yang sudah berjalan.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	db	DatabaseConnection	Atribut privat pada SistemKRSManager untuk menyimpan objek abstraksi database.
2	printer	CetakKRSManager	Atribut privat pada SistemKRSManager untuk menyimpan objek layanan pencetakan dokumen KRS.
3	mahasiswa	String	Parameter fungsi untuk menyimpan nama mahasiswa yang sedang memproses KRS.
4	matakuliah	String	Parameter fungsi untuk menyimpan nama atau daftar mata kuliah yang diambil mahasiswa.
5	jalur	kalkulasiJalurUKT	Parameter fungsi untuk menentukan strategi kalkulasi biaya berdasarkan jalur masuk mahasiswa.

6	totalUKT	Double	Variabel lokal untuk menyimpan hasil perhitungan nominal biaya kuliah mahasiswa.
7	adaKesalahanFormatKop	Boolean	Variabel kontrol simulasi di CetakKRSManger untuk mendeteksi kesalahan format dokumen cetak.
8	biaya	Double	Atribut privat dan <i>final</i> di dalam <i>enum</i> KalkulasiJalurUKT untuk menyimpan tarif biaya mutlak tiap jalur.
9	nama	String	Atribut privat di kelas MataKuliahTeori dan MataKuliahPraktikum untuk identitas nama mata kuliah.
10	mhs	String	Variabel lokal di kelas Main yang menampung data nama mahasiswa untuk simulasi uji coba.
11	matkul	String	Variabel lokal di kelas Main yang menampung data mata kuliah simulasi.
12	mysql	DatabaseConnection	Variabel objek di kelas Main untuk menjalankan instansiasi database relasional MySQL.
13	nosql	DatabaseConnection	Variabel objek di kelas Main untuk menjalankan instansiasi database awan Cloud NoSQL.
14	skenarioLama	SistemKRSManger	Objek pengontrol utama di kelas Main untuk menjalankan simulasi pengisian KRS sistem lama.
15	skenarioBaru	SistemKRSManger	Objek pengontrol utama di kelas Main untuk menjalankan simulasi pengisian KRS tuntutan baru.

III. Constructor dan method

No	Nama Metode	Jenis metode	Fungsi
1	SistemKRSManager(...)	Constructor	Menginisialisasi nilai awal objek pengelola KRS dengan menyuntikkan ketergantungan komponen database dan printer dari luar.
2	validasiPrasyaratMataKuliah(...)	Functional (return value)	Melakukan pengecekan kelayakan akademis mahasiswa untuk mengambil suatu mata kuliah tertentu.
3	prosesKRS(...)	Procedural (void)	Mengatur alur bisnis utama pengisian KRS, mulai dari validasi, kalkulasi biaya UKT, simpan database, hingga cetak dokumen.
4	cetakDraftKRS(...)	Procedural (void)	Memproses pembuatan dokumen cetak draf resmi KRS berbentuk PDF serta melakukan simulasi validasi format dokumen.
5	simpanKRS(...)	Procedural (void)	Metode abstrak (<i>interface</i>) atau konkret (<i>override</i>) untuk menangani urusan teknis penyimpanan data transaksi mahasiswa ke database.
6	KalkulasiJalurUKT(...)	Constructor	Menginisialisasi nominal tarif biaya mutlak ke setiap konstanta jalur masuk mahasiswa di dalam <i>enum</i> .
7	getBiaya()	Functional (return value)	Mengambil nilai nominal angka biaya kuliah yang terikat pada pilihan strategi jalur masuk mahasiswa.
8	registrasiKelas()	Procedural (void)	Metode abstrak atau konkret untuk mendaftarkan dan memproses data administrasi kelas dari suatu mata kuliah ke dalam sistem.
9	MataKuliahTeori(...)	Constructor	Menginisialisasi nama identitas untuk pembuatan objek mata kuliah berbasis pemahaman teori.
10	MataKuliahPraktikum(...)	Constructor	Menginisialisasi nama identitas untuk pembuatan objek mata kuliah berbasis praktik laboratorium.
11	alokasiAsistenLab()	Procedural (void)	Mengatur dan menjadwalkan alokasi asisten laboratorium khusus untuk mata kuliah praktikum.

12	cekPeralatanPraktikum()	Procedural (void)	Melakukan pemeriksaan kelayakan teknis alat-alat laboratorium sebelum kegiatan praktikum dimulai.
13	main(...)	Procedural (void)	Titik masuk utama (<i>entry point</i>) program untuk mengorkestrasi, merangkai objek, dan mendemonstrasikan pengujian berbagai skenario sistem.

IV. Dokumentasi dan Pembahasan Code

a. SistemKRSManger.java

```
public class SistemKRSManger {
    private final DatabaseConnection db;
    private final CetakKRSManger printer;

    public SistemKRSManger(DatabaseConnection db, CetakKRSManger printer) {
        this.db = db;
        this.printer = printer;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | X
    public boolean validasiPrasyaratMataKuliah(String mahasiswa, String matakuliah) {
        System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
        return true;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | X
    public void prosesKRS(String mahasiswa, String matakuliah, KalkulasiJalurUKT jalur) {
        validasiPrasyaratMataKuliah(mahasiswa, matakuliah);

        double totalUKT = jalur.getBiaya();
        System.out.println("Total UKT Terhitung = Rp " + totalUKT);

        db.simpanKRS(mahasiswa, matakuliah);

        printer.cetakDraftKRS(mahasiswa, matakuliah);
    }
}
```

Kelas SistemKRSManger ini sebagai sumber utama dalam proses pengisian KRS. jika dalam penerapan solid, kelas ini menerapkan SRP dan DIP yang sebelumnya tergabung dengan CetakKRSManger, sekarang hanya berfokus ke alur bisnis proses KRS.

Kelas ini memiliki 2 atribut yaitu **db** dan **printer**, dengan lewat konstruktor “public SistemKRSManger(DatabaseConnection db, CetakKRSManger printer)” kelas ini menerapkan teknik *Dependency Injection*. Objek di database dan printer di masukan dari luar saat kelas ini dibuat. Jadi kelasnya tidak terikat dan terlihat kaku pada satu jenis database atau printer tertentu.

Metode **validasiPrasyaratMataKuliah** berfungsi untuk memastikan bolehkah mahasiswa yang bersangkutan memenuhi syarat untuk mengambil mata kuliah tersebut. Saat ini, fungsi tersebut mengembalikan nilai true sebagai penanda validasi berhasil.

Metode **prosesKRS** adalah pusat dari proses KRS. karena metode ini sangat mengandalkan kerjasama antar objek-objek lain untuk menyelesaikan empat tahapan secara berurutan yaitu melakukan validasi mata kuliah, menghitung total biaya UKT mahasiswa secara dinamis lewat parameter KalkulasiJalurUKT jalur, menyimpan data transaksi KRS ke dalam database, dan mencetak draf KRS untuk mahasiswa.

b. CetakKRSManager.java

```
public class CetakKRSManager {  
    Windsurf: Refactor | Explain | Generate Javadoc | X  
    public void cetakDraftKRS(String mahasiswa, String matakuliah) {  
        System.out.println(x: "Proses Cetak PDF...(load data)");  
        boolean adaKesalahanFormatKop = true;  
        if (adaKesalahanFormatKop) {  
            System.out.println(x: "Format Kop surat gagal! Proses cetak PDF berhenti.");  
            return;  
        }  
        System.out.println(x: "Draft PDF berhasil dicetak.");  
    }  
}
```

Class ini untuk melakukan tugas mencetak dokumen KRS . pemisahan kelas ini salah satu implementasi dari SRP, dengan memisahkan fungsi cetak ke dalam file yang berbeda meminimalisir perubahan kecil pada format kop surat PDF yang bisa merusak algoritma penyimpanan database mahasiswa.

Metode cetakDraftKRS menerima parameter String mahasiswa dan String matakuliah untuk mencetak draf dokumen resmi KRS. Simulasi penanganan error dalam variabel **adaKesalahanFormatKop** yang di set ke nilai true. Ini adalah sebuah simulasi pengkondisian (*defensive programming*) untuk mendeteksi kesalahan format. Jika terjadi error pada format dokumen, sistem akan langsung menghentikan proses cetak (return;) demi keamanan data, tanpa memengaruhi atau mengganggu jalurnya transaksi data yang sudah berhasil disimpan sebelumnya. Jika seluruh pemeriksaan format aman, sistem akan menyelesaikan proses dengan mencetak pesan "Draft PDF berhasil dicetak."

c. CloudNoSQLDatabaseConnection.java

```
public class CloudNoSQLDatabaseConnection implements DatabaseConnection {  
    Windsurf: Refactor | Explain | Generate Javadoc | X  
    @Override  
    public void simpanKRS(String mahasiswa, String matakuliah) {  
        System.out.println(x: "Menghubungkan ke Cloud NoSQL Server (Scalable)...");  
        System.out.println("KRS Mahasiswa " + mahasiswa + " dengan kelas [" + matakuliah + "] sukses masuk NoSQL Document.");  
    }  
}
```

Kelas ini bertugas untuk menangani penyimpanan data KRS ke infrastruktur basis data modern cloud. File ini dibuat untuk menerapkan salah satu konsep SOLID yaitu DIP, karena rektorat meminta untuk memindahkan dari MySQL lama ke Cloud NoSQL.

d. DatabaseConnection.java

```
public interface DatabaseConnection {  
    void simpanKRS(String mahasiswa, String matakuliah);  
}
```

Kelas ini berguna untuk interface yang bertindak sebagai pembatas antara logika bisnis aplikasi dengan urusan teknis penyimpanan data. Pembuatan file ini untuk menerapkan DIP agar sistem SIAKAD yang sebelumnya kaku.

e. MySQLDatabaseConnection.java

```
public class MySQLDatabaseConnection implements DatabaseConnection {  
    Windsurf: Refactor | Explain | Generate Javadoc | X  
    @Override  
    public void simpanKRS(String mahasiswa, String matakuliah) {  
        System.out.println(x: "Menghubungkan ke MySQL Database Server Kampus...");  
        System.out.println("KRS Mahasiswa " + mahasiswa + " dengan kelas [" + matakuliah + "] sukses masuk tabel MySQL.");  
    }  
}
```

Kelas ini yang bertugas menangani penyimpanan data KRS ke dalam tabel basis data relasional MySQL. Bareng dengan kelas NoSQL yang baru, file ini untuk penerapan DIP dan LSP di dalam sistem. File ini dibuat untuk menunjukan bahwa arsitektur ini sangat fleksibel, karena sistem lama tidak perlu langsung dibuang secara ekstrem, melainkan tetap dipertahankan sebagai opsi koneksi yang valid selama masa transisi perpindahan infrastruktur.

f. KalkulasiJalurUKT.java

```
public enum KalkulasiJalurUKT {  
    REGULER(biaya: 5000000),  
    KARYAWAN(biaya: 7500000),  
    INTERNASIONAL(biaya: 15000000),  
    BTDIKMISTI(biaya: 0),  
    MBKM(biaya: 4000000);  
  
    private final double biaya;  
  
    KalkulasiJalurUKT(double biaya) {  
        this.biaya = biaya;  
    }  
  
    Windsurf: Refactor | Explain | Generate Javadoc | X  
    public double getBiaya() {  
        return this.biaya;  
    }  
}
```

File **KalkulasiJalurUKT** dibuat menggunakan tipe data **enum** (Enumerasi) karena khusus di Java yang menyimpan daftar konstanta jalur masuk mahasiswa beserta biaya tetapnya masing-masing, dengan menerapkan konsep OCP. Kelas ini berguna untuk menyatukan data biaya ke dalam struktur enum, ratusan baris pernyataan if-else bertumpuk yang tidak berguna di dalam sistem lama dapat dihapus.

g. **OperasiPraktikum.java**

```
public interface OperasiPraktikum {  
    void alokasiAsistenLab();  
    void cekPeralatanPraktikum();  
}
```

Kelas **OperasiPraktikum** adalah sebuah interface yang dibuat untuk menampung operasi-operasi yang hanya berlaku pada jenis mata kuliah praktikum, pembuatan file ini untuk menerapkan konsep ISP dan LSP dalam SOLID.

h. **OperasiMataKuliah.java**

```
public interface OperasiMataKuliah {  
    void registrasiKelas();  
}
```

File **OperasiMataKuliah** adalah interface yang bertindak sebagai kontrak umum untuk semua jenis mata kuliah di kampus. Pembuatan file ini merupakan penerapan konsep SOLID menggunakan ISP.

Pada sistem sebelum menerapkan SOLID, interface ini tercemar karena memaksa semua kelas memiliki fungsi laboratorium. Namun, di bereskan dan kini hanya memuat operasi universal yang hanya dimiliki setiap jenis mata kuliah.

i. **MataKuliahTeori.java**

```
public class MataKuliahTeori implements OperasiMataKuliah {  
    private String nama;  
  
    public MataKuliahTeori(String nama) {  
        this.nama = nama;  
    }  
  
    @Override  
    public void registrasiKelas() {  
        System.out.println("Registrasi Kuliah Teori: " + nama);  
    }  
}
```

Kelas **MataKuliahTeori** merupakan sebuah komponen yang memodelkan jenis mata kuliah tentang pemahaman teori di kampus. Pembuatan file ini merupakan penerapan dari ISP dan LSP.

Pada rancangan kode sebelum menerapkan SOLID, kelas ini terpaksa rusak karena menuliskan metode laboratorium yang tidak sesuai untuk kelas teori. Maka, kelas **MataKuliahTeori** berdiri sendiri untuk system yang lebih aman.

j. **MataKuliahPraktikum.java**

```
public class MataKuliahPraktikum implements OperasiMataKuliah, OperasiPraktikum {
    private String nama;

    public MataKuliahPraktikum(String nama) {
        this.nama = nama;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | X
    @Override
    public void registrasiKelas() {
        System.out.println("Registrasi Praktikum: " + nama);
    }

    Windsurf: Refactor | Explain | Generate Javadoc | X
    @Override
    public void alokasiAsistenLab() {
        System.out.println("Asisten Lab dialokasikan untuk " + nama);
    }

    Windsurf: Refactor | Explain | Generate Javadoc | X
    @Override
    public void cekPeralatanPraktikum() {
        System.out.println("Peralatan Lab diperiksa untuk " + nama);
    }
}
```

Kelas MataKuliahPraktikum merupakan sebuah komponen yang memodelkan mata kuliah yang membutuhkan fasilitas laboratorium di kampus. Pembuatan file ini merupakan penerapan ISP dan LSP secara berdampingan.

k. **Main.java**

```
public class Main {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | X
    public static void main(String[] args) {
        CetakKRSManger printer = new CetakKRSManger();
        String mhs = "Ahmad Zaidan";
        String matkul = "Desain Arsitektur Perangkat Lunak";

        System.out.println(x: "--- DEMO 1: SISTEM LAMA (MYSQL + REGULER) ---");
        DatabaseConnection mysql = new MySQLDatabaseConnection();
        SistemKRSManger skenarioLama = new SistemKRSManger(mysql, printer);
        skenarioLama.prosesKRS(mhs, matkul, KalkulasiJalurUKT.REGULER);

        System.out.println(x: "\n\n");

        System.out.println(x: "--- DEMO 2: TUNTUTAN BARU (CLOUD NOSQL + MBKM) ---");
        DatabaseConnection nosql = new CloudNoSQLDatabaseConnection();
        SistemKRSManger skenarioBaru = new SistemKRSManger(nosql, printer);
        skenarioBaru.prosesKRS(mhs, matkul, KalkulasiJalurUKT.MBKM);
    }
}
```

Kelas Main bertindak sebagai tempat uji coba yang menguji seluruh komponen kecil yang telah dirombak. File ini menyatukan semua file kode menggunakan teknik Dependency Injection dan membuktikan keberhasilan sistem dalam beradaptasi menghadapi perubahan skenario bisnis tanpa merusak kode inti.

Di dalam file ini, sistem berhasil mendemonstrasikan dua simulasi skenario (Skenario Lama vs Skenario Baru) yang berjalan berdampingan.

V. Kesimpulan

Berdasarkan praktikum *refactoring* yang telah dilakukan, dapat disimpulkan bahwa penerapan prinsip SOLID terbukti sangat efektif dalam mengubah sistem SIAKAD menjadi lebih rapi, efisien, dan mudah dirawat. Masalah tanggung jawab ganda dan antarmuka yang memaksa berhasil dibereskan melalui prinsip *Single Responsibility* dan *Interface Segregation*. Dengan memindahkan fungsi cetak ke kelas CetakKRSManger serta memecah antarmuka mata kuliah menjadi OperasiMataKuliah dan OperasiPraktikum, kelas MataKuliahTeori dan MataKuliahPraktikum kini bisa menjalankan fungsinya masing-masing secara natural tanpa memicu *error* paksaan seperti pada sistem lama. Di samping itu, fleksibilitas sistem dalam menghadapi perubahan aturan kampus kini juga meningkat drastis. Penerapan prinsip *Open/Closed* menggunakan struktur *enum* KalkulasiJalurUKT sukses memangkas baris pengondisian if-else yang rumit, sehingga jalur baru seperti MBKM bisa ditambahkan secara mudah tanpa mengganggu logika jalur lama. Terakhir, karena implementasi prinsip *Dependency Inversion* dan *Liskov Substitution* lewat abstraksi DatabaseConnection, kelas utama SistemKRSManger tidak lagi terikat pada satu jenis database aja. Hal ini dibuktikan pada kelas Main, di mana sistem bisa menukar koneksi dari MySQLDatabaseConnection ke CloudNoSQLDatabaseConnection dengan sangat mudah dari luar.

IV. Daftar Pustaka

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikum Pemrograman Berorientasi Objek - Modul 12: SOLID Principles. Program Studi Ilmu Komputer, Universitas Pertamina

Dosen PBO. (2026). SOLID principles. Universitas Pertamina